

MODULUS

Retail Operations Platform

Success Criteria

Version 1.0

1. Purpose

This document defines the complete set of criteria that must be met for the Modulus project to be considered finished. Criteria are grouped by category: product, architecture, engineering quality, performance, CI/CD, E2E, and portfolio presentation.

Every criterion in this document is binary. It either passes or it does not. There is no partial credit. The project is complete when every criterion in every category passes.

2. Product Criteria

These criteria verify that the application does what it is supposed to do from a user perspective.

Criterion	Target	Category
Users can log in with all three defined credential sets	All three pass	Product
Role-based access correctly restricts section navigation per role	Verified for all three roles	Product
Inventory: products can be created, edited, deactivated, and reactivated	All four operations succeed	Product
Inventory: search by name and SKU returns correct filtered results	Verified on seeded data	Product
Inventory: stock badge colour is correct for all three stock ranges	Green, amber, red correct	Product
Inventory: category management allows adding and renaming categories	Both operations succeed	Product
Orders: order table filters correctly by all five status values	All five filters verified	Product
Orders: order detail view shows customer info, line items, and timeline	All three sections render	Product
Orders: valid status transitions succeed; invalid transitions are blocked	State machine enforced in UI and API	Product
Analytics: all four charts render with data from the seeded set	All four charts render	Product
Analytics: date range selector re-fetches and re-renders all charts	Verified on range change	Product
Logout redirects to the login page and clears the auth session	Redirect confirmed, re-	Product

Criterion	Target	Category
	auth required	

3. Architecture Criteria

These criteria verify the micro-frontend architecture is correctly implemented.

Criterion	Target	Category
Each remote application has its own independent Vercel project	4 separate Vercel projects	Architecture
The shell loads remotes dynamically at runtime, not at build time	Verified via network tab: remoteEntry.js fetched on navigation	Architecture
React runs as a single shared instance across all four applications	No duplicate React instance errors in console	Architecture
Deploying a remote does not require redeploying the shell	Verified by deploying inventory independently	Architecture
Error boundary renders fallback UI when a remote URL is unreachable	Verified by pointing to invalid URL	Architecture
Auth context is accessible inside remote components via useAuth hook	Verified: user object accessible in all three remotes	Architecture
Remote URLs are configured via environment variables, not hardcoded	Verified in shell build config	Architecture
Shared packages are consumed via workspace references, not npm publish	Verified in package.json of each app	Architecture

4. Engineering Quality Criteria

These criteria verify that the codebase meets the engineering standards defined in the System Design Document.

Criterion	Target	Category
TypeScript strict mode compiles with zero errors across all workspaces	0 errors on tsc --noEmit	Quality
ESLint produces zero errors across all workspaces	0 errors on turbo run lint	Quality
No any types exist in application code	Verified via no-explicit-any ESLint rule	Quality
No console.log statements exist in committed code	Verified via no-console ESLint rule	Quality
All API response types are sourced from packages/types	No inline API type definitions in app code	Quality
Vitest unit test coverage is above 80 percent for packages/ui and packages/auth	>80% on coverage report	Quality
All selectors in Playwright tests use data-testid attributes	No CSS or XPath selectors in page objects	Quality
Environment variables are validated at startup via Zod	App fails loudly on missing env var	Quality
MSW is initialised only in the shell, not in any remote	Verified: remotes contain no MSW initialisation code	Quality

5. Performance Criteria

These criteria are measured using Lighthouse run against each production Vercel URL in an incognito browser window with no extensions.

Criterion	Target	Category
Shell Lighthouse Performance score	Above 85	Performance
Inventory Lighthouse Performance score	Above 85	Performance
Orders Lighthouse Performance score	Above 85	Performance
Analytics Lighthouse Performance score	Above 85	Performance
Shell Lighthouse Accessibility score	Above 90	Performance

Criterion	Target	Category
Inventory Lighthouse Accessibility score	Above 90	Performance
Orders Lighthouse Accessibility score	Above 90	Performance
Analytics Lighthouse Accessibility score	Above 90	Performance
No layout shift visible during remote load transitions	CLS below 0.1 on shell	Performance
Remote loading skeleton appears within 200ms of navigation	Observed in browser devtools	Performance

6. CI/CD Criteria

These criteria verify that the GitHub Actions pipelines are correctly configured and behave as specified.

Criterion	Target	Category
Four independent workflow files exist, one per application	4 .yaml files in .github/workflows/	CI/CD
A push to apps/shell triggers only shell.yaml, not others	Verified via GitHub Actions run history	CI/CD
A push to apps/inventory triggers only inventory.yaml, not others	Verified via GitHub Actions run history	CI/CD
A deliberate lint error fails the pipeline at the lint stage	Pipeline fails, subsequent stages do not run	CI/CD
A deliberate TypeScript error fails at the typecheck stage	Pipeline fails, subsequent stages do not run	CI/CD
A deliberate failing unit test fails at the unit test stage	Pipeline fails, subsequent stages do not run	CI/CD
Pull request pipelines run all stages except deploy	Verified on a test PR	CI/CD
Branch protection requires all four pipelines to pass before	Merge blocked	CI/CD

Criterion	Target	Category
merge	on any pipeline failure	
Each application deploys to its own distinct Vercel project	Verified in Vercel dashboard	CI/CD
All required secrets are configured in GitHub repository settings	All 8 secrets present	CI/CD

7. E2E Criteria

These criteria verify that the Playwright suite is correctly implemented and integrated into the pipeline.

Criterion	Target	Category
All 14 test steps pass on the production composed application	14/14 pass in Playwright report	E2E
A deliberately broken test step fails the shell pipeline	Pipeline marked failed on test failure	E2E
HTML report publishes to GitHub Pages after every pipeline run	Report URL accessible after each run	E2E
Report URL is unique per pipeline run (includes run ID in path)	Different URL per run verified	E2E
Slack alert fires on test failure with all required fields	Alert received with test name, MFE, report URL	E2E
Slack alert does not fire on a passing run	Verified: no alert on clean run	E2E
All selectors in test files use page object model methods only	No raw selectors in spec files	E2E
Test constants file defines all test data identifiers	No hardcoded IDs in spec files	E2E

8. Portfolio Presentation Criteria

These criteria verify that the project is presentable to the target audience: freelance clients, startup engineers, contract role interviewers, and large company hiring managers.

Criterion	Target	Category
All four production URLs are live and accessible without authentication	URLs load without error	Portfolio
README at monorepo root covers: overview, architecture, local setup, env vars, production URLs	All five sections present	Portfolio
Architecture diagram is included in the README	Diagram present and legible	Portfolio
All six documentation files are complete and present in the repository	6 docx files committed	Portfolio
Portfolio website card is updated with project name, stack, and live links	Card visible on portfolio site	Portfolio
LinkedIn project entry is written and published	Entry visible on LinkedIn profile	Portfolio
Screen capture walkthrough of the full user journey is recorded	Recording available for portfolio use	Portfolio
Project can be explained in one sentence that a non-technical client understands	Sentence defined and tested	Portfolio
Project can be explained in three sentences that a senior engineer respects	Technical summary defined	Portfolio

9. One-Sentence and Three-Sentence Descriptions

9.1 For Non-Technical Clients

Modulus is an internal operations dashboard that lets retail staff manage products, process orders, and monitor sales performance — built so that each team can update and deploy their section of the system independently without affecting the others.

9.2 For Senior Engineers

Modulus is a micro-frontend platform built on Webpack 5 Module Federation, decomposed into four independently deployable React 18 applications sharing a singleton auth context and a shared Tailwind-based component library via a Turborepo monorepo.

Each application has its own GitHub Actions CI/CD pipeline running lint, typecheck, Vitest unit tests, build, and Vercel deployment in sequence, with the shell pipeline triggering a Playwright E2E suite after deploy.

Test failures block the pipeline, publish an HTML report to GitHub Pages at a run-scoped URL, and fire a Slack alert containing the failed test name, the affected MFE, and a direct link to the report.